TEACHING WALLAH

Teaching Exam

Subject – Computer Science

Data Structure and Algorithm

Lecture No -3

By-Satyendra Chaudhary Sir

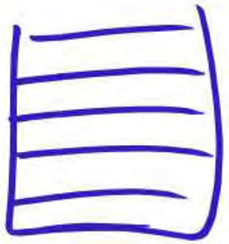
Topics to be Covered

Topic

- Complexities.
- Data structure.
  - Array
  - Stack
  - Recursion
  - Binary Tree

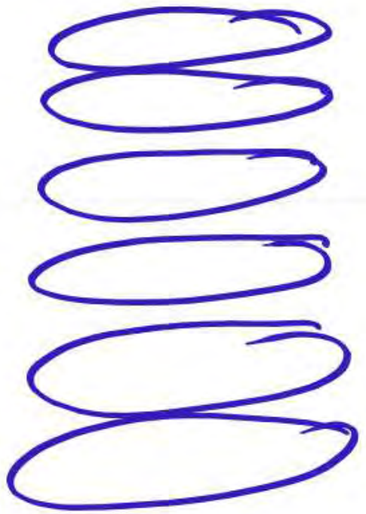
: Introduction to Stack

- Definition: Linear data structure following LIFO (Last In First Out)
- Real-life Examples:
  - Pile of plates



Array

  - Undo feature in editors

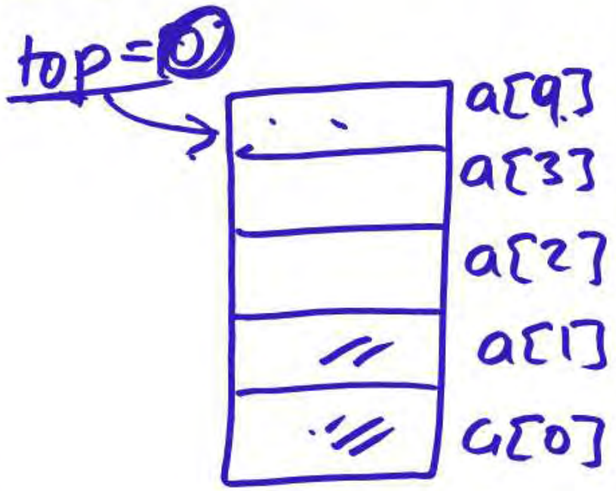


: Stack Operations

- **Push:** Insert element at the top
- **Pop:** Remove element from the top
- **Peek/Top:** View the top element
- **isEmpty():** Check if stack is empty

Algorithm for Push Operation

1. Check if the stack is full.
2. If not, increment `top`
3. Insert element at `stack[top]`.



Pseudocode:

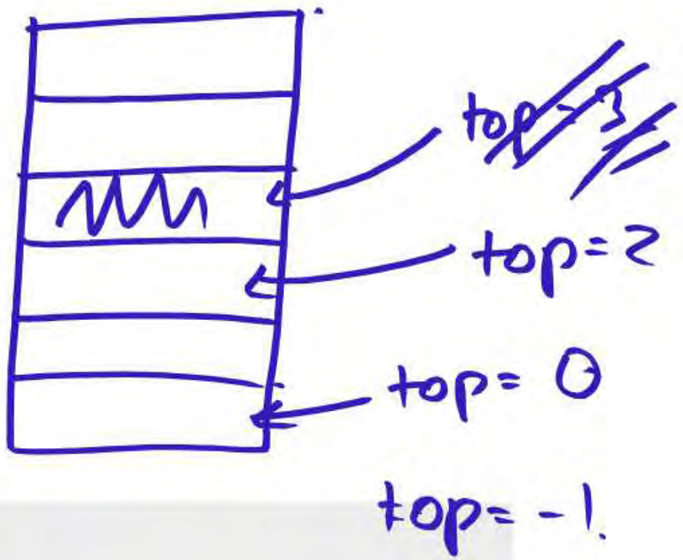
```
if top == MAX - 1:
    print("Overflow")
else:
    top = top + 1
    stack[top] = value
```

a[6]

TEACHING WALLAH

: Algorithm for Pop Operation

1. Check if the stack is empty
2. If not, remove element from `stack[top]`
3. Decrement `top`



Pseudocode:

```
if top == -1:
    print("Underflow")
else:
    value = stack[top]
    top = top - 1
```

Stack Implementation (Array)

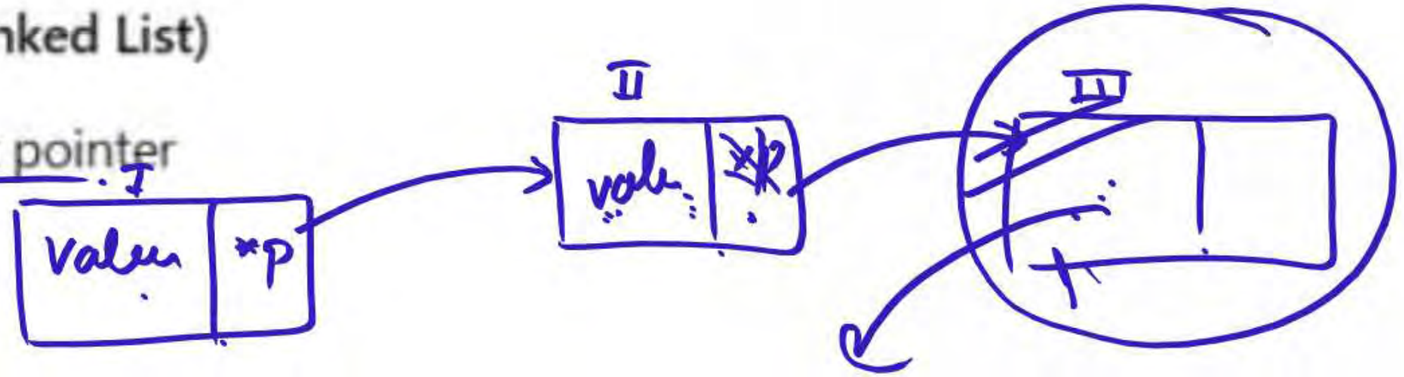
- Use an array and `top` index
- Fixed size

Code Snippet (C-style):

```
int stack[SIZE], top = -1;
```

': Stack Implementation (Linked List)

- Use nodes with data and next pointer
- Dynamic size



Advantages:

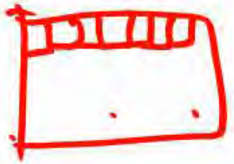
- No fixed size
- Efficient memory usage

: Advantages of Stack

- Easy to implement
- Used in function calls, expression evaluation → arithmetic
- Manages memory (recursive calls)

: Disadvantages of Stack

- Limited size in array implementation
- Can cause stack overflow
- Complex to implement in some cases



TEACHING WALLAH

: Infix to Prefix Conversion

- Use stack to reverse and convert
- Steps:
  1. Reverse the infix expression
  2. Swap (and)
  3. Convert to postfix
  4. Reverse result → prefix

a+b] → Infix.

operand operator.

reverse polish notation

ab+] - postfix notation.

+ab] → prefix notation.

polish ,,

$2 \times 8 \times 5 = 80$

$a \times b = b \times a$. $- + a * b c$. $\wedge d \wedge e f$.

Postfix Notation

$a + b \times c - d \wedge e \wedge f$

a b c × + d e f ^ ^ −.

1) a b c × + d e f ^ ^ −.

2) a b c × + d e ^ f ^ −.

3) a b + c × d − e ^ f ^

$27 \div 81 \div 9$

$a \div b \neq b \div a$.

left associative

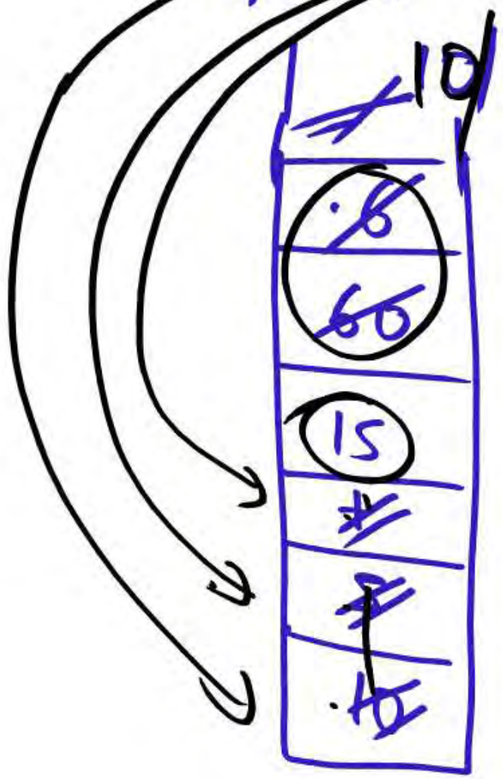
only ^ are right associative

precedence order

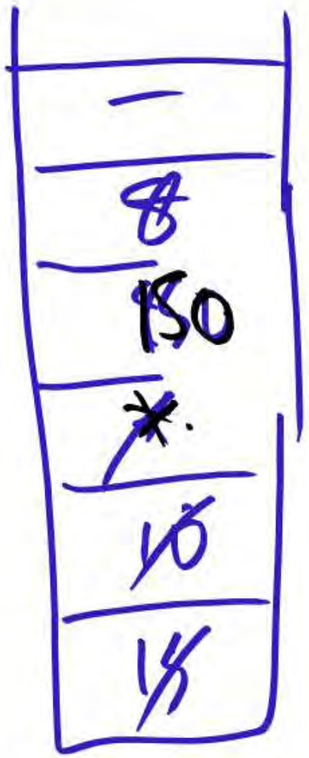
^
÷
×
+
−

postfix →

← prefix



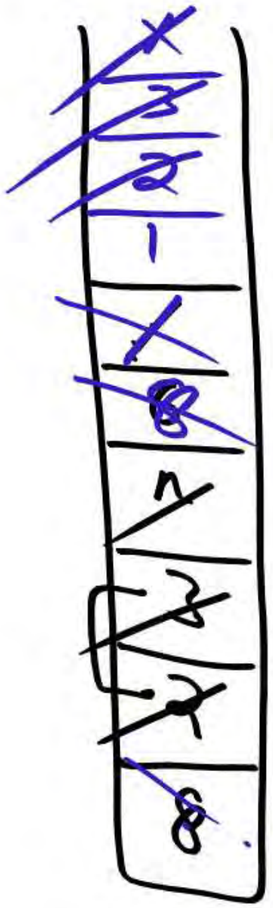
60 / 6 = 10



150

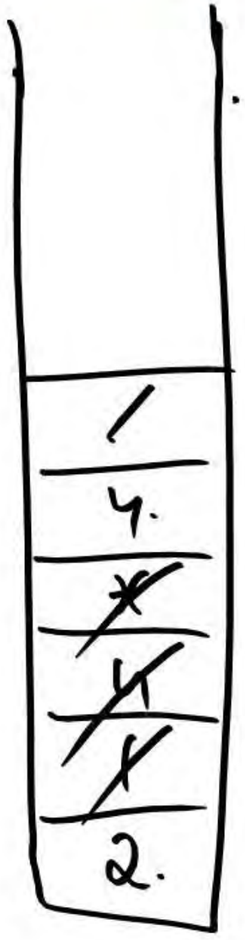
142

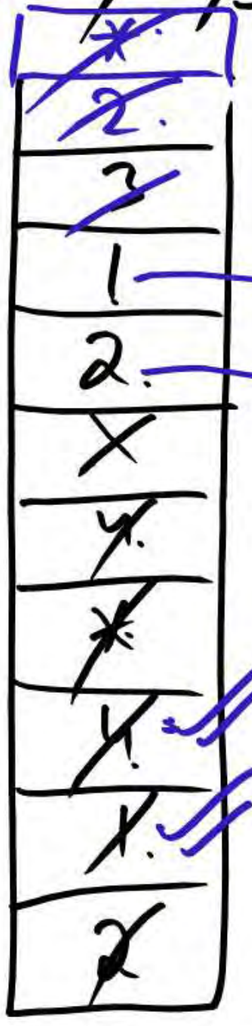
8 2 3 * / 2 3 * + 5 1 * -



(2)

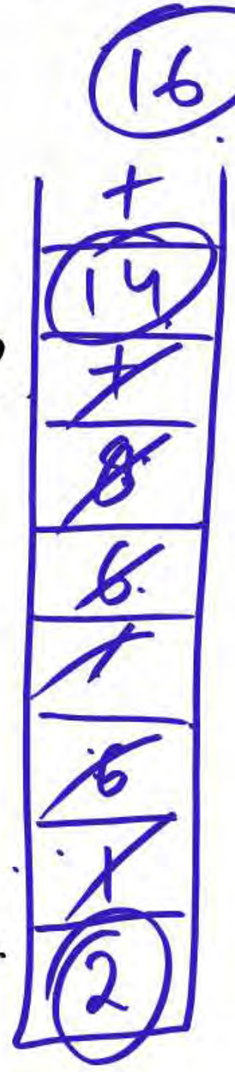
++8/*2231/*41=7





1*4=4.

9/2=2



6÷!

I: Infix to Postfix Conversion

- Steps:
  1. Scan from left to right
  2. If operand, add to output
  3. If operator, pop higher precedence from stack
  4. Push current operator

1. Process of inserting an element in stack is called _____

a) Create

b) Push ✓

c) Evaluation

d) Pop

2. Process of removing an element from stack is called _____

a) Create

b) Push

c) Evaluation

d) Pop

3. In a stack, if a user tries to remove an element from an empty stack it is called _____

a) Underflow

b) Empty collection

c) Overflow

d) Garbage Collection

4. Pushing an element into stack already having five elements and stack size of 5, then stack becomes _____

a) Overflow

b) Crash

c) Underflow

d) User flow

5. Entries in a stack are "ordered". What is the meaning of this statement?

a) A collection of stacks is sortable ✗

b) Stack entries may be compared with the '<' operation ✗

c) The entries are stored in a linked list ✗

d) There is a Sequential entry that is one by one ✓

6. Which of the following is not the application of stack?

a) A parentheses balancing program

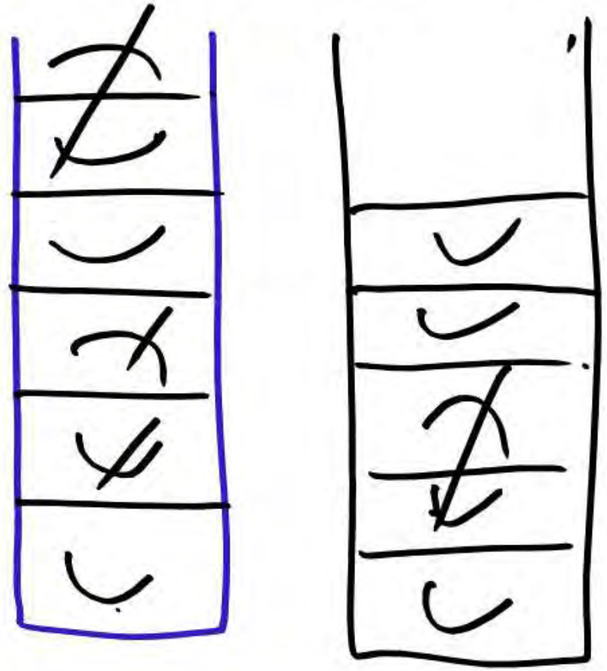
b) Tracking of local variables at run time

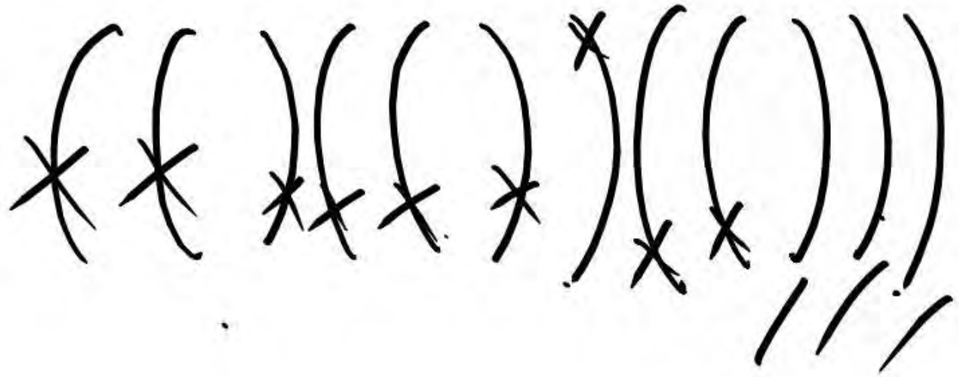
c) Compiler Syntax Analyzer

d) Data Transfer between two asynchronous process

7. Consider the usual algorithm for determining whether a sequence of parentheses is balanced. The maximum number of parentheses that appear on the stack AT ANY ONE TIME when the algorithm analyzes: (()(()))(()))?

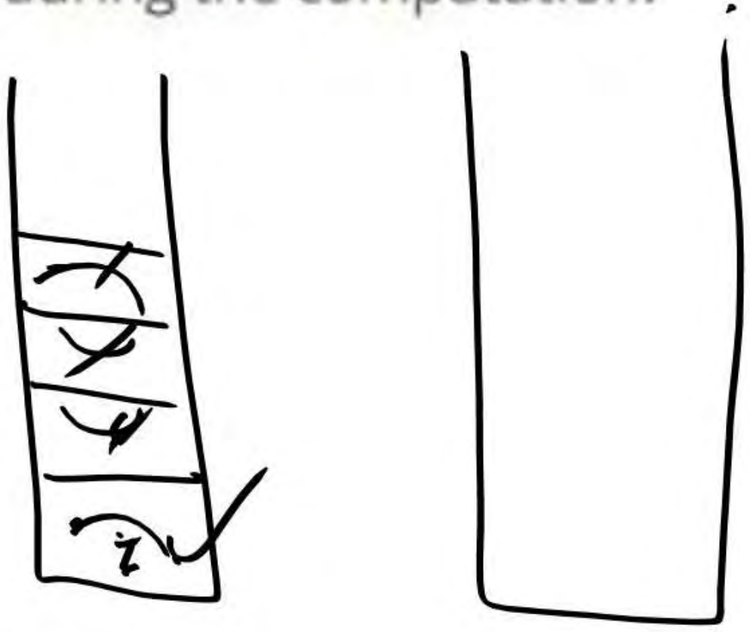
a) 1
b) 2
c) 3
d) 4 or more





8. Consider the usual algorithm for determining whether a sequence of parentheses is balanced. Suppose that you run the algorithm on a sequence that contains 2 left parentheses and 3 right parentheses (in some order). The maximum number of parentheses that appear on the stack AT ANY ONE TIME during the computation?

a) 1
b) 2
c) 3
d) 4 or more



9. What is the value of the postfix expression 6 3 2 4 + – *?

a) 1

b) 40

c) 74

d) -18

10. Here is an infix expression: 4 + 3*(6*3-12). Suppose that we are using the usual stack algorithm to convert the expression from infix to postfix notation. The maximum number of symbols that will appear on the stack AT ONE TIME during the conversion of this expression?

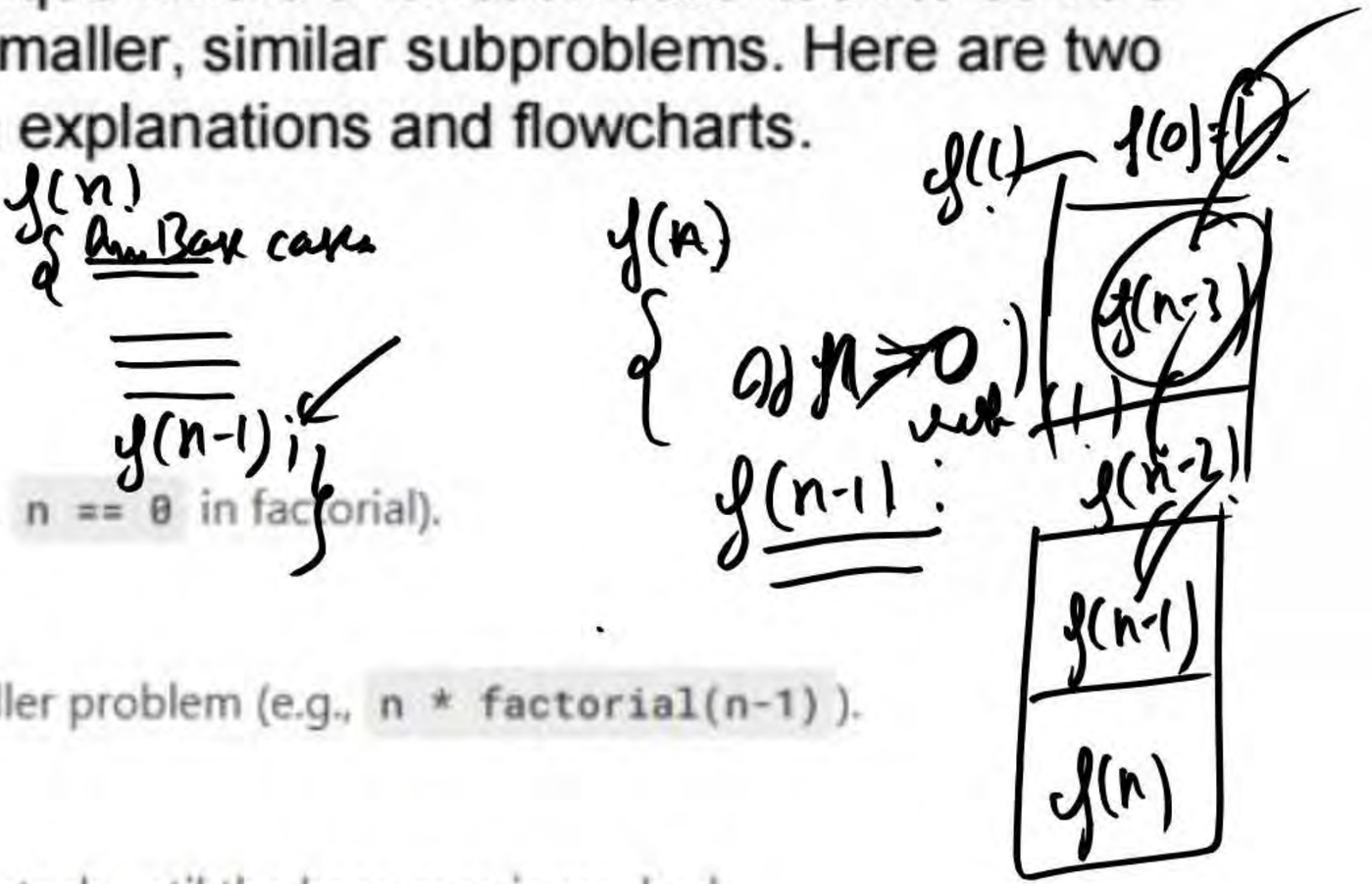
a) 1

b) 2

c) 3

d) 4

Recursion is a programming technique where a function calls itself to solve a problem by breaking it down into smaller, similar subproblems. Here are two easy-to-understand examples with explanations and flowcharts.



Key Concepts of Recursion:

1. **Base Case:**
   - A condition to stop recursion (e.g., `n == 0` in factorial).
2. **Recursive Case:**
   - The function calls itself with a smaller problem (e.g., `n * factorial(n-1)`).
3. **Call Stack:**
   - Each recursive call is added to the stack until the base case is reached.
4. **Unwinding:**
   - The stack resolves backwards once the base case is met.

0! = 1
1! ⇒ 1
2! = 2×1
3! = 3×2!

Example 1: Factorial Calculation

The factorial of a number `n` (denoted as `n!`) is the product of all positive integers up to `n`.

Recursive Definition:

- `n! = n × (n-1)!`
- Base Case: `0! = 1`

f(3);
return: 3× f(2)
return: 2 × f(1)
return: 1 × f(0)
1
..

Python Code:

```python
def factorial(n):
    if n == 0:  # Base case
        return 1
    else:
        return n * factorial(n - 1)  # Recursive call

print(factorial(3))  # Output: 6 (3! = 3 × 2 × 1)
```

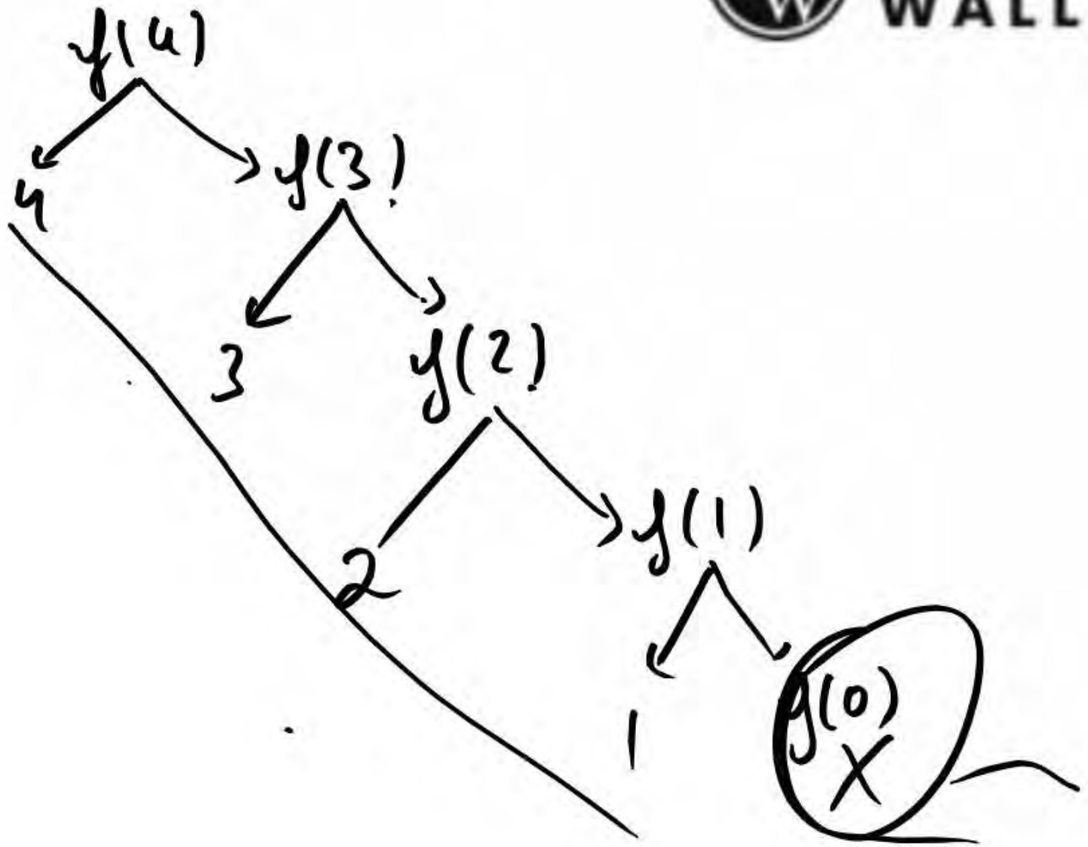
Execution Flow (for `factorial(3)`):

1. Call `factorial(3)`
   - `3 != 0` → `return 3 × factorial(2)`
2. Call `factorial(2)`
   - `2 != 0` → `return 2 × factorial(1)`
3. Call `factorial(1)`
   - `1 != 0` → `return 1 × factorial(0)`
4. Call `factorial(0)`
   - `0 == 0` → `return 1` (Base case stops recursion).
5. Unwinding the Calls:
   - `factorial(1) = 1 × 1 = 1`
   - `factorial(2) = 2 × 1 = 2`
   - `factorial(3) = 3 × 2 = 6` → Final result.

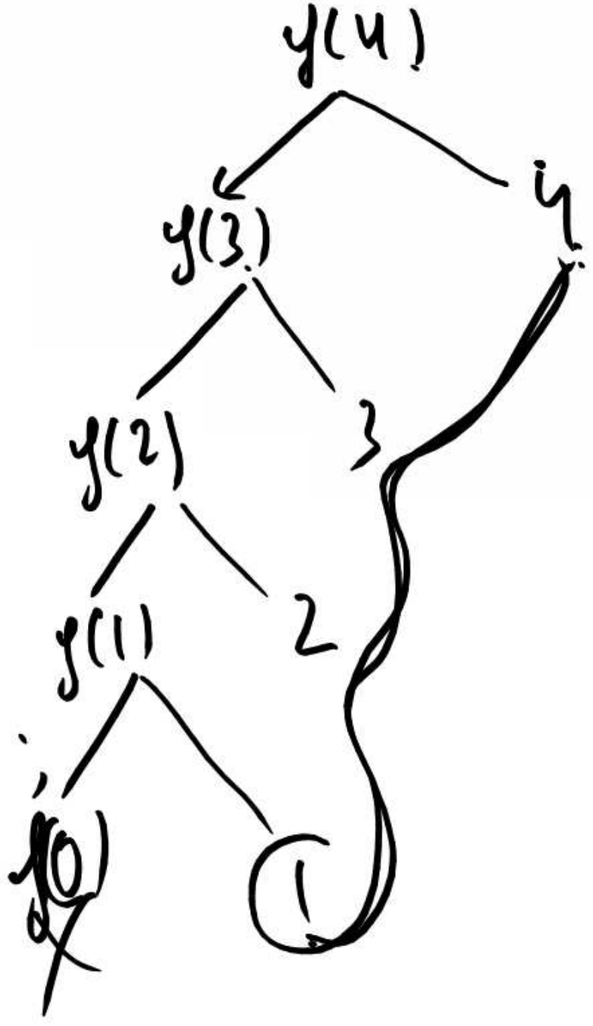
```
factorial(3)
   ↓
3 * factorial(2)
       ↓
    2 * factorial(1)
           ↓
        1 * factorial(0)
               ↓
            1 (Base Case)
```

```
fun(4);

void fun(int n)
{ if (n>0)
    { print("%d", n);
      fun(n-1);
    }
}
```

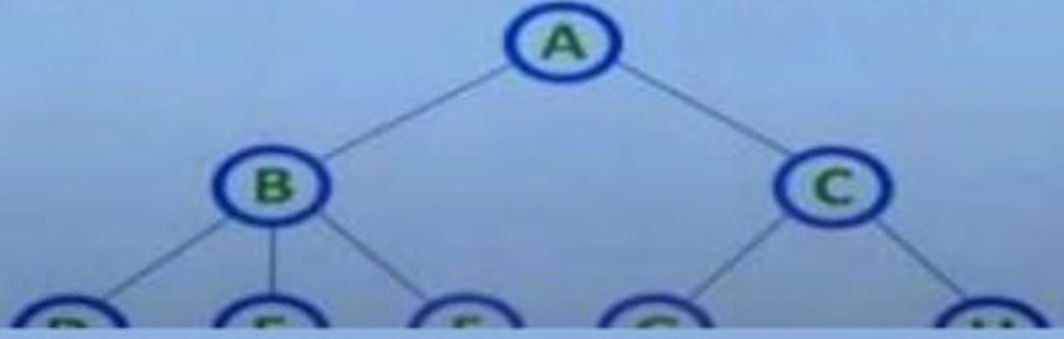


```
fun(4);
void fun(int n)
{
  if(n>0)
    {
      fun(n-1);
      printf("%d", n);
    }
}
```



Data Structure	Access	Insert	Delete	Use Case
Array	O(1)	O(n)	O(n)	Random access, fixed size
Linked List	O(n)	O(1)/O(n)	O(1)/O(n)	Dynamic size, insertion-heavy
Stack	O(1)	O(1)	O(1)	Recursion, parsing
Queue	O(1)	O(1)	O(1)	Task scheduling
Hash Table	O(1)*	O(1)*	O(1)*	Fast lookup by key
Tree (BST)	O(log n)	O(log n)	O(log n)	Sorted data, hierarchy
Heap	O(n)	O(log n)	O(log n)	Priority handling
Graph	Varies	Varies	Varies	Network modeling
Trie	O(L)	O(L)	O(L)	Prefix search

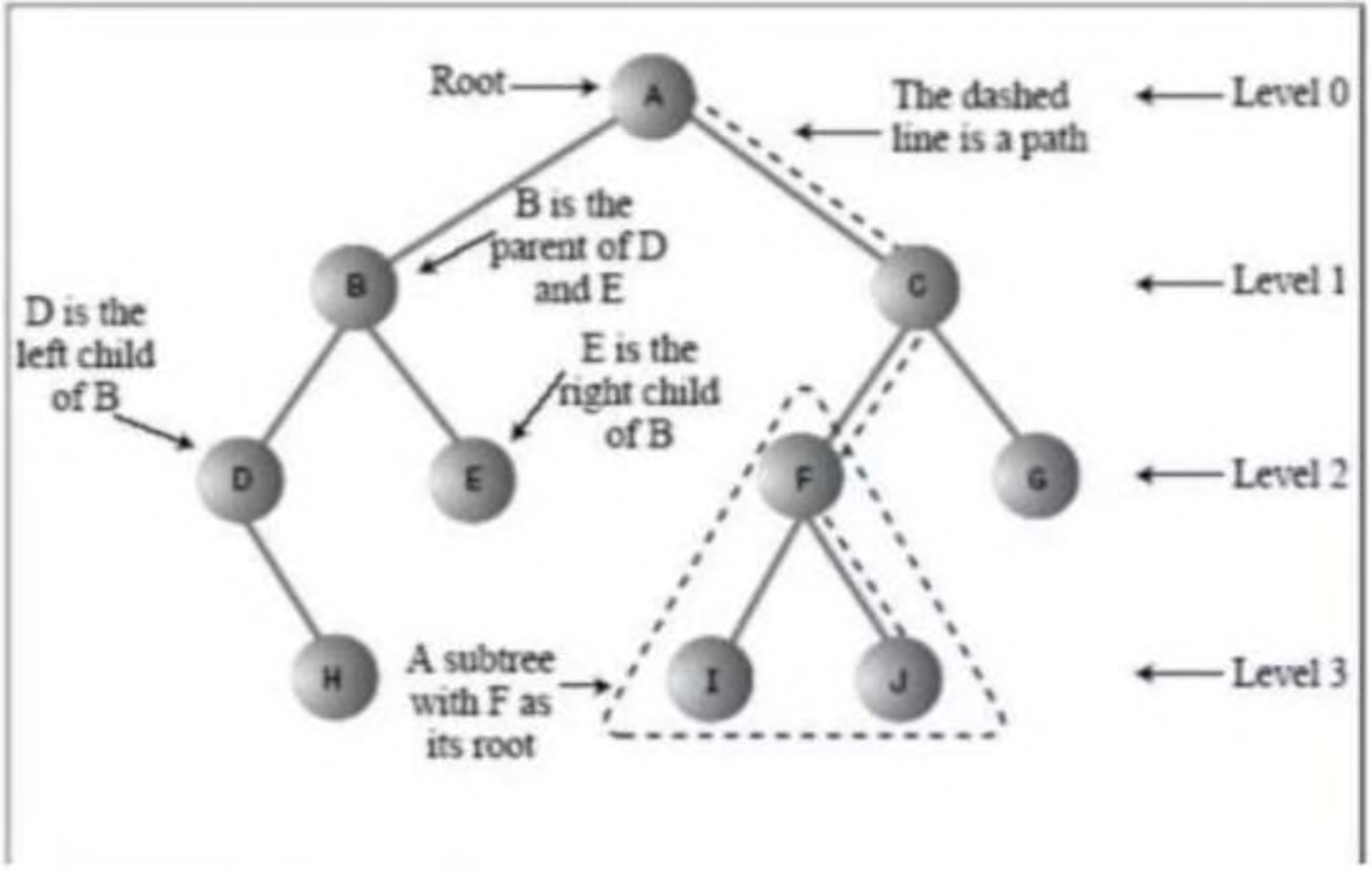
- **Root Node**: The top-most node, serving as the origin of the tree.
- **Edge**: The connection between two nodes. A tree with 'N' nodes has exactly 'N-1' edges.
- **Parent Node**: A node that has one or more children.
- **Child Node**: A node that descends from a parent node.
- **Leaf (External) Node**: A node with no children.
- **Internal Node**: A node with at least one child.
- **Degree of Node**: The number of children a node has.
- **Level/Depth**: Indicates the step count from the root (Level 0) to a particular node.
- **Path**: A sequence of nodes connected by edges.
- **Subtree**: A tree formed from a child node and its descendants.

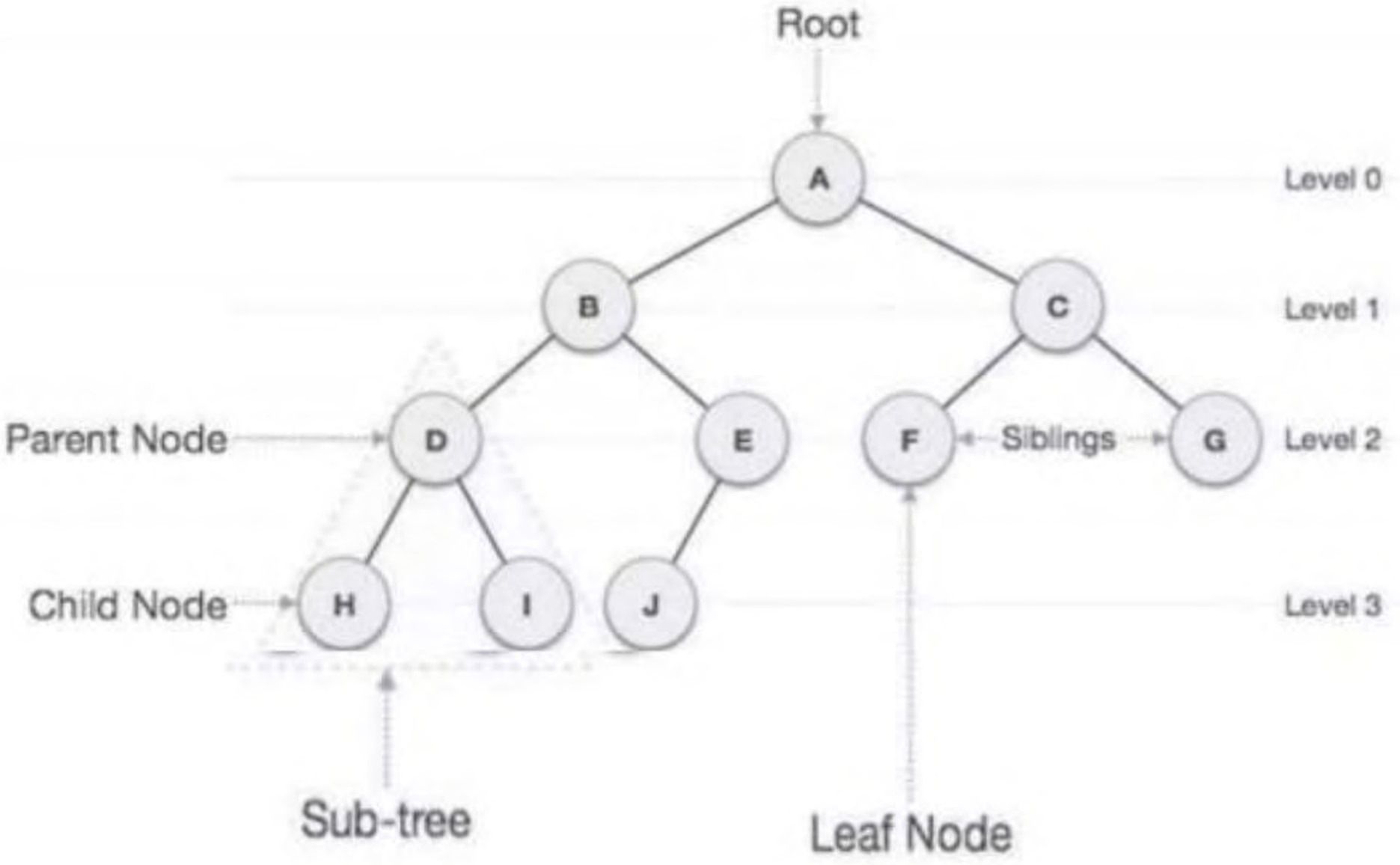


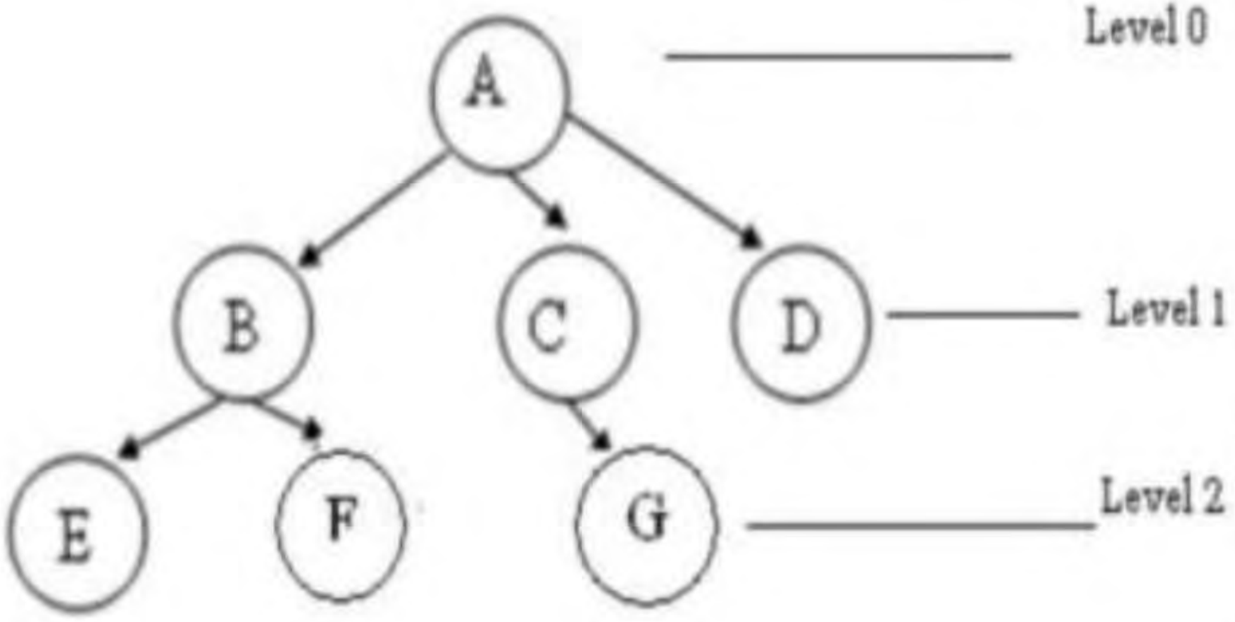
Binary tree

- A **binary tree** is a type of tree data structure where each node can have at most **two children**, commonly referred to as **left** and **right** child nodes. It is structured as a set of nodes with the following properties:
- **Empty Tree:** A binary tree can be empty (no nodes).
- **Root Node:** If not empty, the tree has a unique root node (R).
- **Subtrees:** The remaining nodes are partitioned into two disjoint subtrees, **T1** (left) and **T2** (right).
- Representation of tree in memory
  - Sequential representation – using an array info and left child and right child
  - Linked Representation – using self-referential structure node

```
struct node {
   int data;
   struct node* left;
   struct node* right;
}
```







- ✔ A is the root node
- ✔ B is the parent of E and F
- ✔ D is the sibling of B and C
- ✔ E and F are children of B
- ✔ E, F, G, D are external nodes or leaves
- ✔ A, B, C are internal nodes
- ✔ Depth of F is 2
- ✔ the height of tree is 2
- ✔ the degree of node A is 3
- ✔ The degree of tree is 3

Characteristics of trees

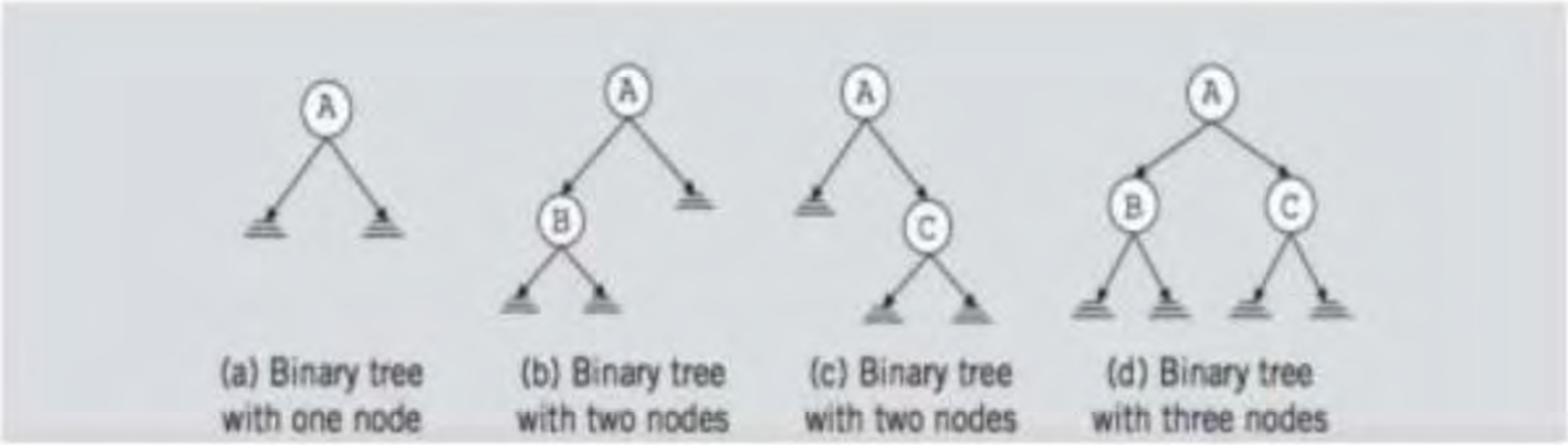
- Non-linear data structure
- Combines advantages of an ordered array
- Searching as fast as in ordered array
- Insertion and deletion as fast as in linked list
- Simple and fast

Application

- Directory structure of a file store
- Structure of an arithmetic expressions
- Used in almost every 3D video game to determine what objects need to be rendered.
- Used in almost every high-bandwidth router for storing router-tables.
- used in compression algorithms, such as those used by the .jpeg and .mp3 file-formats.

Binary Trees

- A **binary tree**, *T*, is either empty or such that
  - *I.* *T* has a special node called the **root** node
  - *II.* *T* has two sets of nodes L_T and R_T, called the **left subtree** and **right subtree** of *T*, respectively.
  - *III.* L_T and R_T are binary trees.

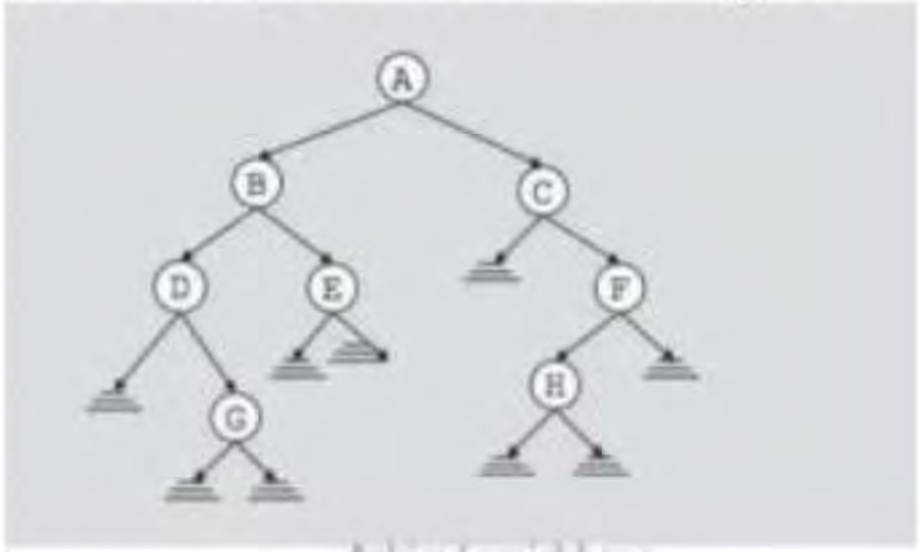


Binary Tree

- The root node of this binary tree is A.

- The left sub tree of the root node, which we denoted by L_A, is the set L_A = {B,D,E,G} and the right sub tree of the root node, R_A is the set R_A={C,F,H}

- The root node of L_A is node B, the root node of R_A is C and so on



Binary Tree Properties

- If a binary tree contains **m** nodes at level **L**, it contains at most **2m** nodes at level **L+1**

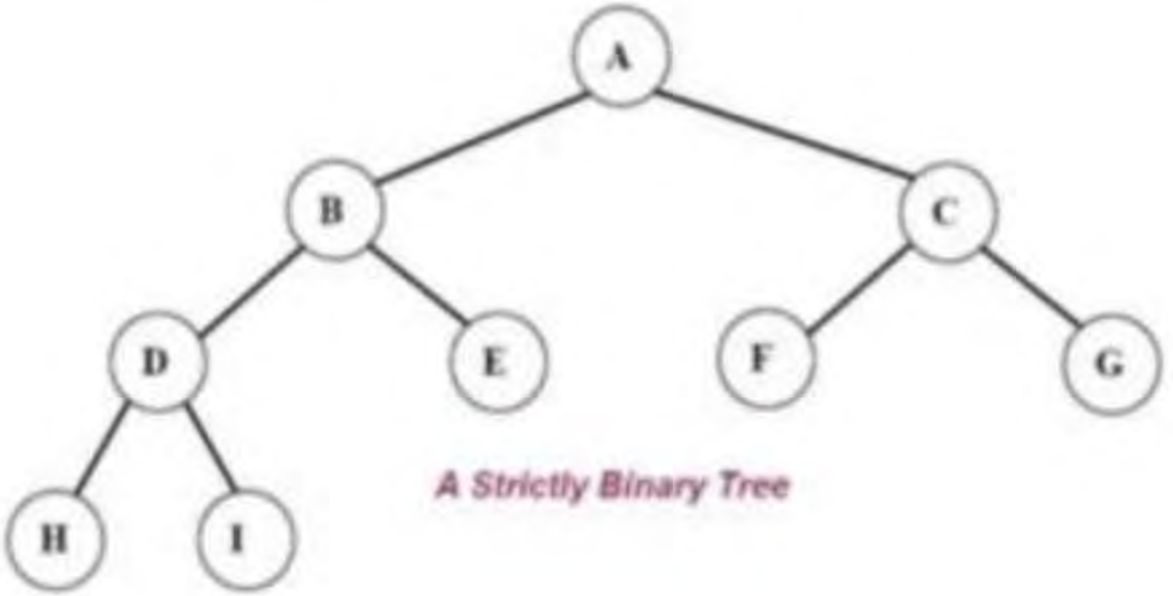
- Since a binary tree can contain at most 1 node at level 0 (the root), it contains at most **2L** nodes at level L.

Types of Binary Tree

- Complete binary tree
- Strictly binary tree
- Almost complete binary tree

- **Or, to put it another way,** all of the nodes in a strictly binary tree are of degree zero or two, never degree one.

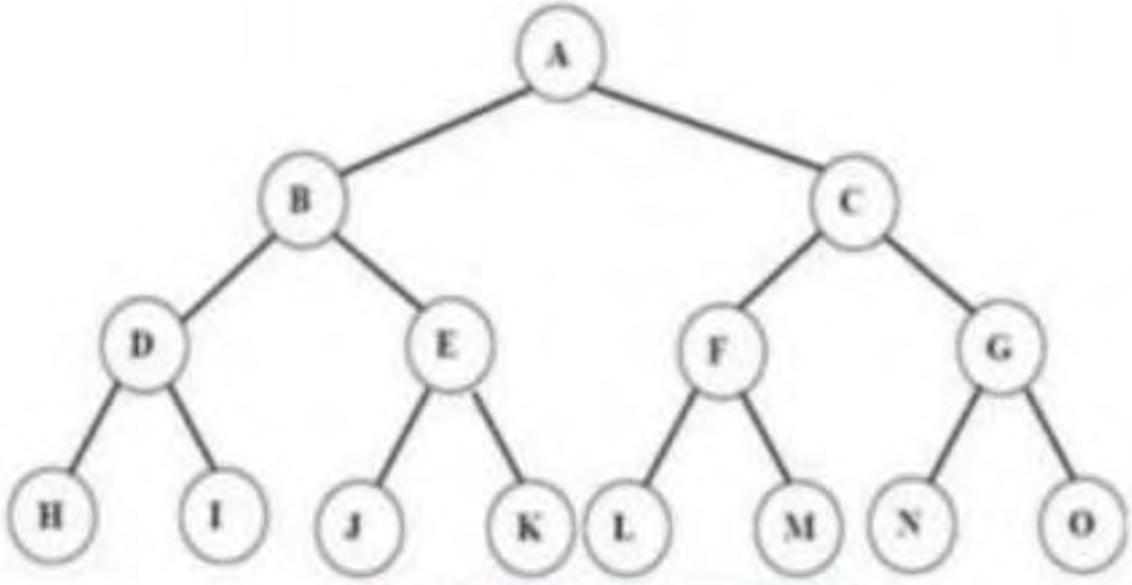
- A strictly binary tree with N leaves always contains 2N – 1 nodes.



A Strictly Binary Tree

- A ***complete binary tree*** of depth **d** is called strictly binary tree if all of whose leaves are at level **d**.

- A complete binary tree has 2^d nodes at every depth **d** and 2^d -1 non leaf nodes



A complete Binary Tree of depth 3

- An almost complete binary tree is a tree where for a right child, there is always a left child, but for a left child there may not be a right child.

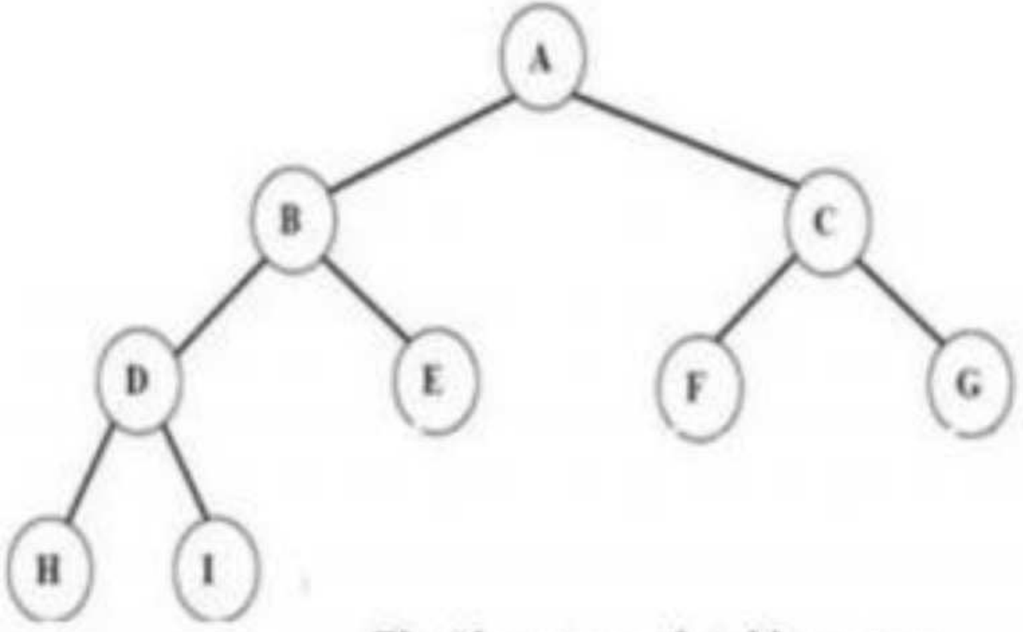


Fig Almost complete binary tree.

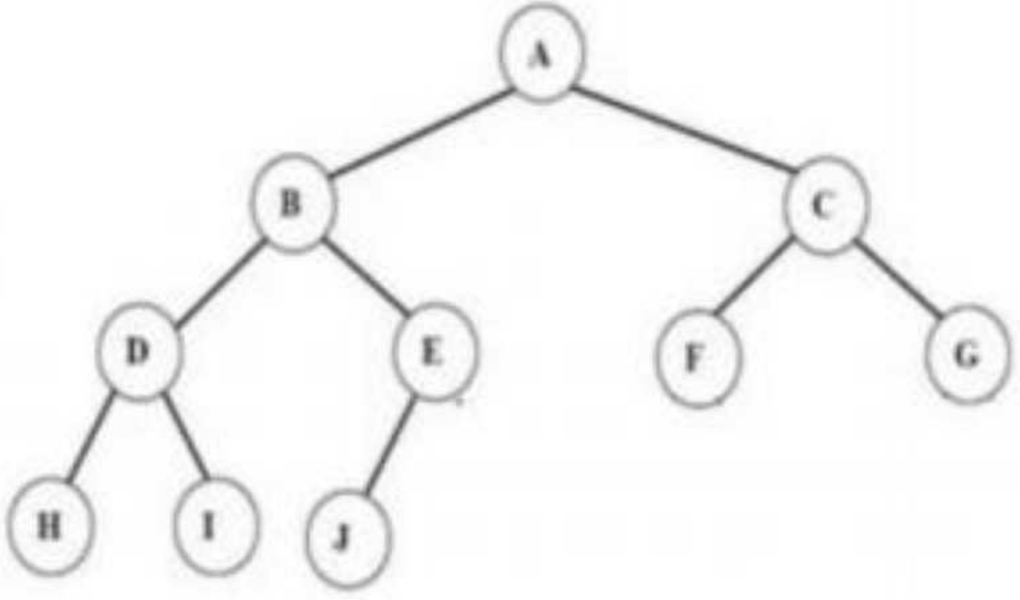


Fig Almost complete binary tree but not strictly
Since node E has a left son but not a right son.

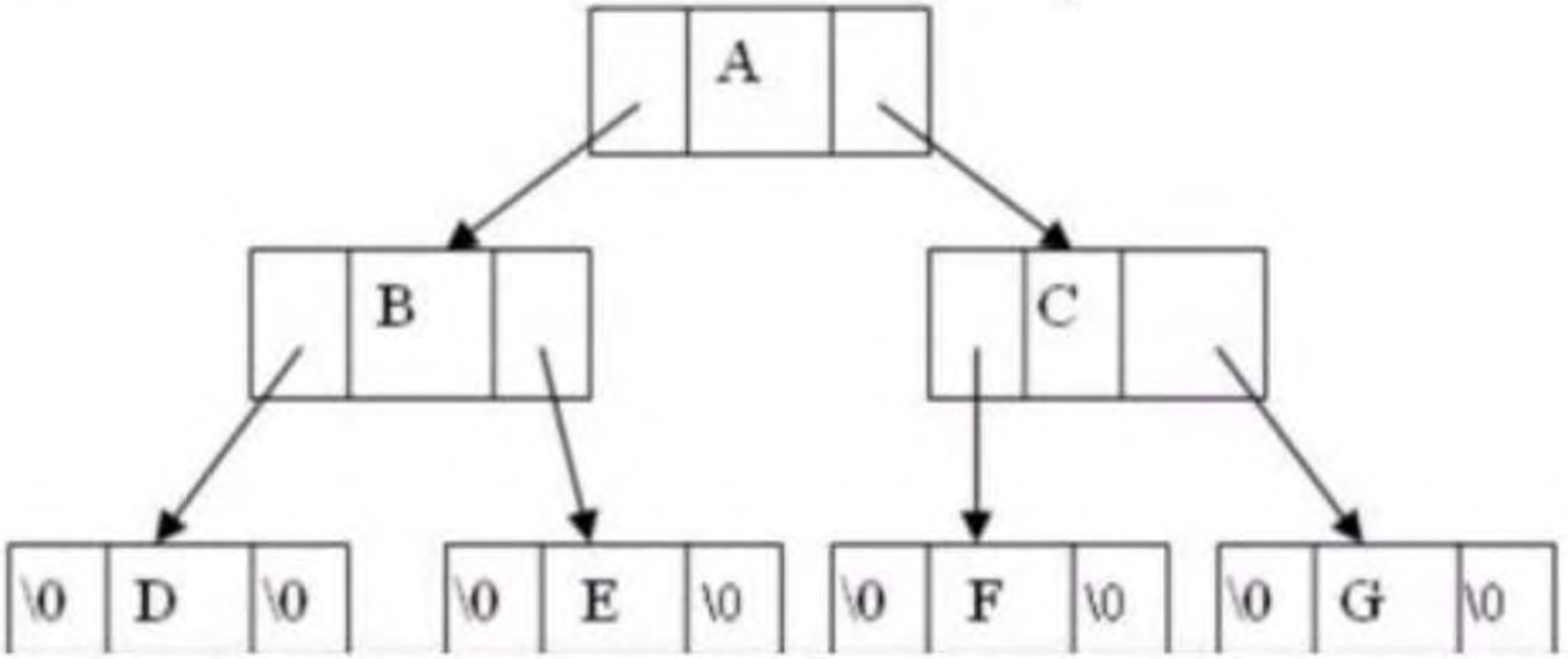
Operations on Binary tree:

- ✔ **father(n,T):**Return the parent node of the node n in tree T. If n is the root, NULL is returned.
- ✔ **LeftChild(n,T):**Return the left child of node n in tree T. Return NULL if n does not have a left child.
- ✔ **RightChild(n,T)**:Return the right child of node n in tree T. Return NULL if n does not have a right child.
- ✔ **Info(n,T):** Return information stored in node n of tree T (ie. Content of a node).
- ✔ **Sibling(n,T):** return the sibling node of node **n** in tree T. Return NULL if n has no sibling.

- ✔ **Root(T):** Return root node of a tree if and only if the tree is nonempty.
- ✔ **Size(T):** Return the number of nodes in tree T
- ✔ **MakeEmpty(T):** Create an empty tree T
- ✔ **SetLeft(S,T):** Attach the tree S as the left sub-tree of tree T
- ✔ **SetRight(S,T):** Attach the tree S as the right sub-tree of tree T.
- ✔ **Preorder(T):** Traverses all the nodes of tree T in preorder.
- ✔ **postorder(T):** Traverses all the nodes of tree T in postorder
- ✔ **Inorder(T):** Traverses all the nodes of tree T in inorder.

C representation for Binary tree:

```
struct bnode
{
        int info;
        struct bnode  *left;
        struct bnode  *right;
};
struct bnode *root=NUI
```



Tree traversal

- Traversal is a process to visit all the nodes of a tree and may print their values too.

- All nodes are connected via edges (links) we always start from the root (head) node.

- There are three ways which we use to traverse a tree
  - In-order Traversal
  - Pre-order Traversal
  - Post-order Traversal

- Pre-order

<root><left><right>

- In-order

<left><root><right>

- Post-order

<left><right><root>

- The preorder traversal of a nonempty binary tree is defined as follows:
  - Visit the root node
  - Traverse the left sub-tree in preorder
  - Traverse the right sub-tree in preorder

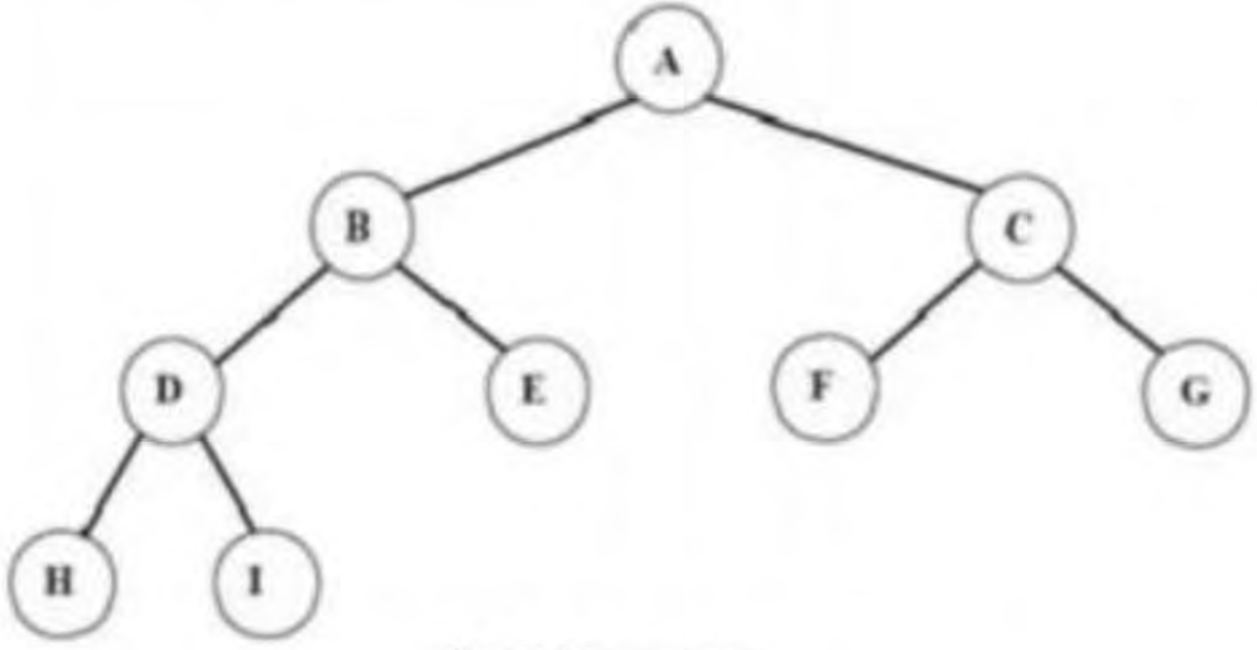
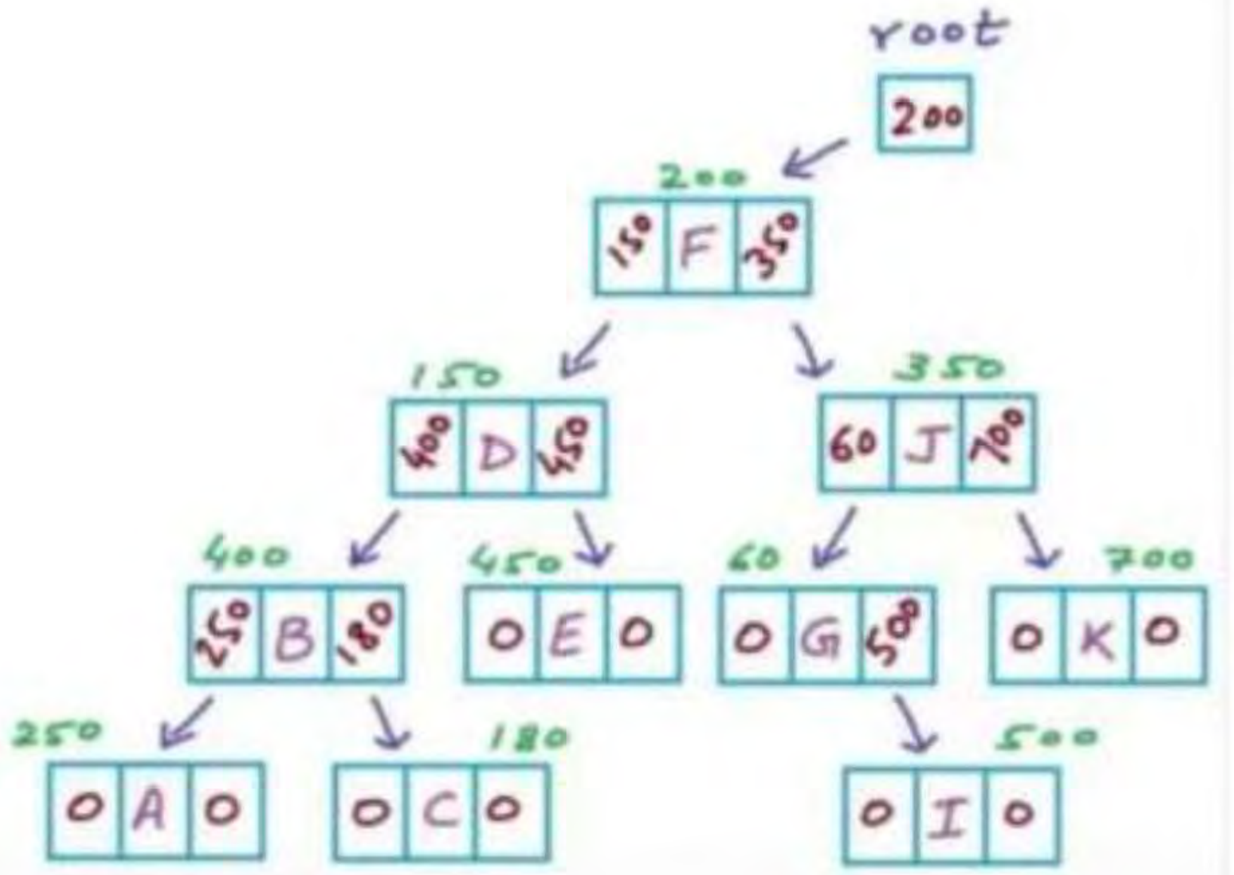


fig Binary tree

The preorder traversal output of the given tree is: A B D H I E C F G

Pre-order Pseudocode

```
struct Node{
    char data;
    Node *left;
    Node *right;
}
void Preorder(Node *root)
{
    if (root==NULL) return;
    printf ("%c", root->data);
    Preorder(root->left);
     Preorder(root->right);
 }
```



- The in-order traversal of a nonempty binary tree is defined as follows:
  - Traverse the left sub-tree in in-order
  - Visit the root node
  - Traverse the right sub-tree in inorder

- The in-order traversal outpu of the given tree is
  H D I B E A F C G

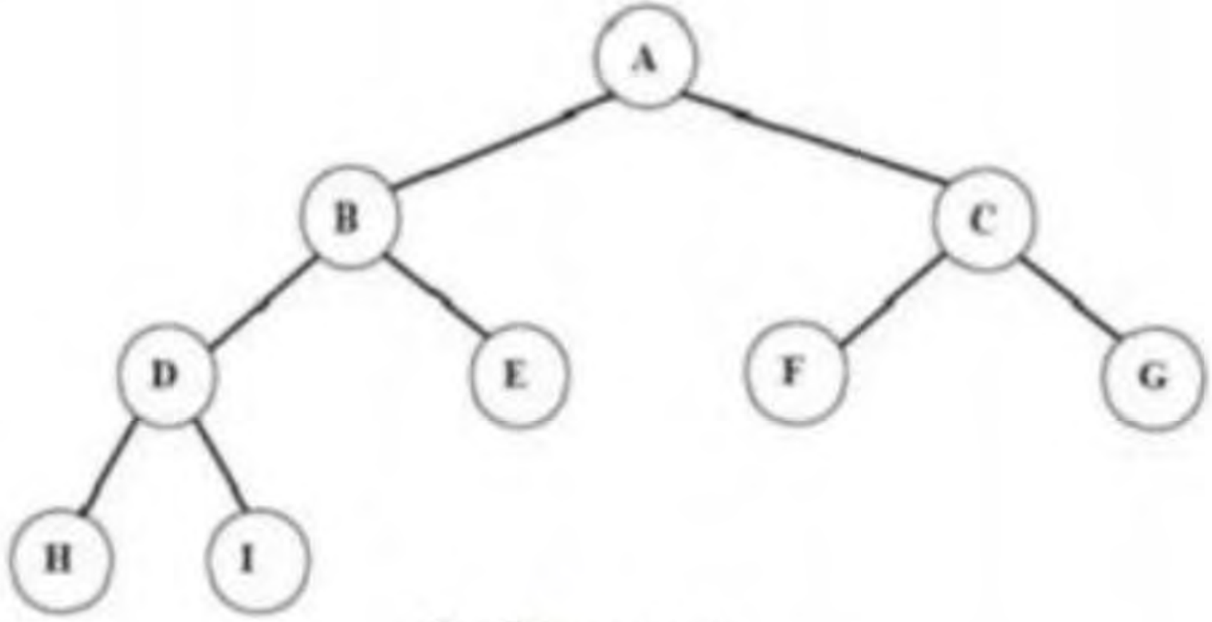


fig Binary tree

2 mins Summary

TEACHING WALLAH

Topic

Data Structure and Algorithm

TEACHING WALLAH

THANK YOU